

Sustainable In-Orbit Analytics: Energy-Aware Scheduling for LEO Earth-Observation Constellations

Anonymous Author(s)

Abstract

Earth observation is essential for time-sensitive applications such as disaster response, environmental monitoring, and large-scale situational awareness. In-orbit analytics strengthens these services by allowing satellites to process sensed data onboard, thereby reducing response latency and dependence on downlink opportunities. Long-term in-orbit analytics in low Earth orbit Earth observation constellations are constrained by continuously changing visibility and energy conditions. As satellites move along their orbits, observation opportunities, task deadlines, and solar charging cycles evolve together, making the scheduling of observation jobs inherently inter-temporal. In this paper, we first propose a unified discrete-time model that brings together periodic observation jobs, release-deadline constraints, satellite-target visibility, onboard computation, and battery evolution in a single framework, thereby exposing the inter-temporal dependencies that conventional slot-wise schedulers fail to capture. Based on this model, we then develop *Energy-Carryover Flow Scheduling (ECoFlow)*, which formulates the scheduling problem as a maximum-flow problem on a time-expanded graph and explicitly embeds harvested energy and battery carryover through energy nodes and carryover edges. This allows the scheduler to reason globally over the planning horizon instead of making myopic slot-by-slot decisions. Evaluations using two-line element propagated orbital traces and representative multi-area observation workloads show that ECoFlow consistently improves the fraction of observation demand completed (i.e., coverage ratio) over baseline methods, with gains of up to 12% under diverse energy, computation, and timing settings.

Keywords

Orbital edge computing, LEO Earth observation, Energy-aware scheduling, Time-expanded max-flow, Sustainable in-orbit analytics

1 Introduction

In many real-world scenarios, such as disaster response, weather monitoring, and traffic surveillance, large geographic areas must be sensed continuously and revisited periodically to provide timely situation updates [1]. Compared with platforms such as Unmanned Aerial Vehicles (UAVs), satellite constellations are better suited to these missions because they can provide wide-area coverage, regular revisit opportunities, and operation over remote or infrastructure-limited regions without requiring local deployment [2]. Traditionally, Earth-observation systems often rely on downloading sensed data to Ground Stations (GSs) for later processing. However, for time-sensitive applications, this download-and-process pipeline is constrained by intermittent ground contact, limited downlink bandwidth, and the large volume of raw observation data, all of which can delay actionable responses [3]. The recent rise of Orbital Edge Computing (OEC) addresses this limitation by enabling satellites to process sensed data onboard and transmit compact

analytics results instead of raw data, thereby reducing end-to-end latency and downlink burden while improving persistent situational awareness over large geographic regions [4]. However, once computing is pushed into orbit, satellites must execute sensing and analytics under tightly constrained onboard energy, storage, and processing resources, while their orbital motion continuously changes target visibility, communication opportunities, and solar charging conditions. These coupled dynamics make long-term sustainable scheduling a key research challenge in satellite-based Earth observation systems [5].

Existing studies on satellite scheduling and real-time resource management have made important progress in task allocation, communication-computation coordination, and deadline-aware optimization [6]. Nevertheless, existing real-time scheduling studies have typically assumed stable and predictable computing resources, where task feasibility is independent of time-varying energy supply [7]. Such assumptions do not hold for satellite systems, in which solar-energy harvesting and communication visibility evolve dynamically along the orbit. To address this gap, we propose *Energy-Carryover Flow Scheduling (ECoFlow)*, a unified scheduling framework for sustainable in-orbit analytics that jointly captures periodic monitoring jobs, time-varying visibility, onboard computation, and orbit-dependent energy evolution, enabling global reasoning over the planning horizon. Our main contributions are as follows:

- We present ECoFlow, a unified scheduling framework for sustainable in-orbit analytics in LEO Earth-observation constellations under coupled timing, visibility, computation, and energy constraints.
- We design a time-expanded maximum-flow formulation for ECoFlow that explicitly captures harvested energy and inter-temporal battery carryover.
- We evaluate ECoFlow using Two-Line Element (TLE) propagated orbital traces and multi-area monitoring workloads, and show up to 12% improvement over baseline methods.

The remainder of this paper is organized as follows. Section 2 presents the system model. Section 3 introduces the proposed max-flow-based scheduling framework and algorithm. Section 4 provides the experimental evaluation. Section 5 reviews the background and related work. Section 6 concludes the paper.

2 System Model and Problem

Figure 1 illustrates the overall scenario considered in this work. We study sustainable in-orbit analytics for a Low Earth Orbit (LEO) Earth-observation constellation to enable long-term monitoring of geographically distributed areas of interest. Here, each task corresponds to a user request to repeatedly observe a specific ground region and deliver updated results within a required time window; for example, a user may request that a flood-prone river basin be revisited every 6 hours and that updated water-extent analysis be returned within 30 minutes of each observation. Each task generates

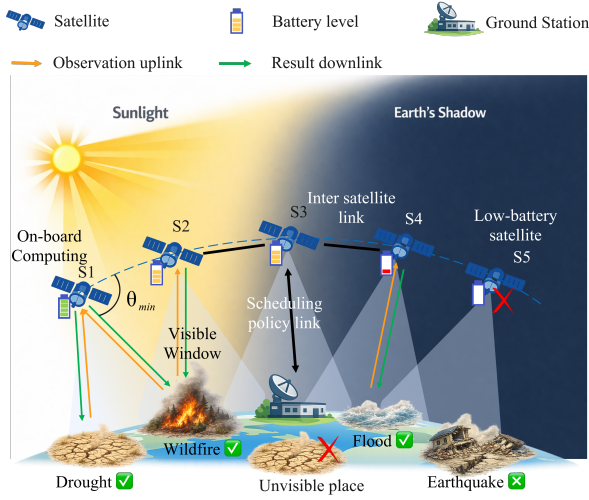


Figure 1: Overview of the sustainable in-orbit analytics system.

monitoring jobs over time, and each job represents one required observation-and-analysis instance. These jobs must be served under coupled constraints from target visibility, onboard computation capability, downlink opportunity, and satellite energy dynamics, including task-dependent energy consumption and time-varying solar charging opportunities. As satellites move along their orbits, these conditions evolve dynamically: satellites in sunlight can harvest solar energy, whereas satellites in the Earth's shadow must rely on stored battery energy to sustain sensing, computing, and communication operations. Therefore, the key problem is how to coordinate sensing, onboard analytics, and result delivery over time under jointly time-varying visibility and energy constraints, so as to support timely and reliable Earth observation services.

2.1 Time Model

We adopt a discrete-time model for the LEO satellite system [8], with time divided into slots $\mathcal{T} = t_k \mid k = 1, \dots, N_t$. This abstraction matches the discrete nature of scheduling decisions, and we assume that visibility, energy, and computation conditions remain constant within each slot. In each slot t_k , a satellite updates its energy state by harvesting solar power when illuminated or by drawing from its battery when in eclipse, and may execute mission operations such as image acquisition and onboard processing, both of which incur energy consumption.

2.2 Workload Model

We consider a set of ground-monitoring tasks $\mathcal{K} = \{\kappa_1, \dots, \kappa_{N_c}\}$, where each task corresponds to an Earth-observation service for a specific geographic area. For example, a task may require a coastal region to be revisited every several time slots and updated analysis results to be produced after each revisit. We characterize each task κ_i by $\kappa_i = (l_i, H_i)$, where l_i denotes the target location and $H_i > 0$ denotes the job release interval measured in time slots.

Over a finite planning horizon of N_t time slots, task κ_i generates a set of jobs, denoted by $\mathcal{M}_i$. We consider all jobs released within the look-ahead window used for scheduling, so that future job arrivals,

visibility opportunities, and energy evolution can be optimized jointly rather than myopically slot by slot. Accordingly, the number of jobs generated by task κ_i is $|\mathcal{M}_i| = \left\lfloor \frac{N_t}{H_i} \right\rfloor$, which becomes N_t/H_i when N_t is divisible by H_i .

Each job in $\mathcal{M}_i$ is denoted by $\zeta_{i,m} = (D_{i,m}, C_{i,m})$, where $D_{i,m} > 0$ is the relative deadline and $C_{i,m} > 0$ is the processing workload measured in CPU cycles. Thus, each task represents a persistent monitoring demand for a geographic region, whereas the jobs in $\mathcal{M}_i$ represent the individual observation-and-analysis instances released over time for that task.

2.3 Satellite Model

We model the satellite constellation as $\mathcal{S} = \{S_1, S_2, \dots, S_{N_s}\}$, where N_s is the number of satellites. Each satellite S_j is characterized by its orbit description TLE_j and battery capacity B_j . Let $b_{j,k}$ denote the available battery energy of satellite S_j at the beginning of slot $t_k \in \mathcal{T}$, with $0 \leq b_{j,k} \leq B_j$. In the current framework, we consider a homogeneous onboard computing setting and assume that all satellites operate at the same nominal computation frequency f . The orbital geometry, together with the resulting sensing and downlink opportunities, is obtained using a satellite-orbit simulator; in this work, we use STK, which is widely adopted in the space industry [9]. The initial battery state $b_{j,1}$, together with updated system parameters and task-set information, is obtained from telemetry at the beginning of the planning horizon.

2.4 Coverage and Visibility

Let $\mathcal{A} = \{a_{i,j,k}\}$ be the satellite-task visibility tensor, where $a_{i,j,k} = 1$ if satellite S_j can observe the location of task κ_i during time slot t_k for subsequent analysis, and $a_{i,j,k} = 0$ otherwise. Similarly, $\mathcal{V} = \{v_{j,k}\}$ denotes the satellite-ground visibility matrix, where $v_{j,k} = 1$ if satellite S_j has a downlink connection to any ground station at time t_k . These tensors capture the geometric feasibility constraints derived from STK and serve as the inputs (i.e., constraints) to the subsequent scheduling optimization.

2.5 Computing Model

Each observation task $\kappa_i \in \mathcal{K}$ executed by satellite $S_j \in \mathcal{S}$ consists of three sequential stages: (i) image capturing, (ii) on-board processing, and (iii) results downlinking to the ground station. When κ_i is executed by S_j , the total execution duration is

$$T_{i,j} = T_{i,j}^{\text{cap}} + T_i^{\text{cmp}} + T_{i,j}^{\text{dl}}, \quad (1)$$

where $T_{i,j}^{\text{cap}}$, T_i^{cmp} , and $T_{i,j}^{\text{dl}}$ denote the durations for capturing, computation, and downlink transmission, respectively, and are derived with details below.

Capturing stage. The capturing time $T_{i,j}^{\text{cap}}$ is constrained by the satellite-task visibility relationship $a_{i,j,k}$. Specifically, the image of task κ_i can only be captured when $a_{i,j,k} = 1$, i.e., when satellite S_j is within the coverage region of task κ_i during slot t_k . When the target is visible, we approximate the capture duration by a constant t^{cap} , which represents the fixed sensing overhead required to acquire a usable image, including sensor exposure and attitude-stabilization

time. Thus,

$$T_{i,j}^{\text{cap}} = \begin{cases} t^{\text{cap}}, & \text{if } a_{i,j,k} = 1, \\ \infty, & \text{otherwise.} \end{cases} \quad (2)$$

Computation stage. The on-board processing time for each job depends on the satellite's CPU frequency f and assigned computation workload Γ_i for the satellite (in CPU cycles).

$$T_i^{\text{cmp}} = \frac{\Gamma_i}{f}. \quad (3)$$

This represents the duration required for the satellite to perform inference or analytics on the captured image before transmission.

Downlink stage. After the on-board processing is completed, the result must be transmitted to a ground station. The downlink process is feasible only when the satellite-ground visibility $v_{j,k} = 1$. The propagation delay between the satellite and the ground station is approximated by

$$T_{i,j}^{\text{dl}} = \frac{d_{i,j}}{c}, \quad (4)$$

where $d_{i,j}$ is the instantaneous distance between satellite i and ground station j , and c is the speed of light [10]. Since $T_{i,j}^{\text{dl}}$ is typically on the order of milliseconds, it is negligible compared with sensing and computation times; however, it is retained here for completeness of the end-to-end latency model.

Remark (Job splitting). Note that a job may be assigned to multiple satellites for processing. Each satellite will capture and process a different fraction of the same area. The corresponding processing time of this job on each satellite becomes the full capturing and downlink time, plus a fraction of the assigned workload in CPU time, derived based on the computation frequency of the assigned satellite, and the total processing time of the job is the sum of those over all assigned satellites.

2.6 Observation Decision Model

Let $x_{i,j,k,m} \in [0, 1]$ be the fraction of the m -th sensing job of task κ_i that is assigned to satellite S_j in time slot t_k . A value of $x_{i,j,k,m} = 0$ indicates that no portion of the job is allocated to satellite S_j in slot t_k , whereas a positive value indicates that the corresponding fraction of the job is scheduled for execution by S_j during that slot. To ensure physical feasibility, an assignment is allowed only when satellite S_j can observe task κ_i in slot t_k , i.e., $x_{i,j,k,m} \leq a_{i,j,k}$, for all i, j, k, m . In addition, a job can only be assigned within its release-deadline window. Then, $x_{i,j,k,m}$ must be 0 for any slot $t_k \notin [r_{i,m}, r_{i,m} + D_{i,m}]$, where $r_{i,m}$ is the release time of $\zeta_{i,m}$.

Finally, since each job can be completed at most once, the total fraction assigned to all satellites over all time slots cannot exceed the full job requirement: $\sum_{j \in \mathcal{S}} \sum_{k \in \mathcal{T}} x_{i,j,k,m} \leq 1$, for all i, m . This constraint ensures that the cumulative assigned fraction of each job, aggregated across all satellites and all feasible time slots, does not exceed one complete job.

2.7 Energy Model

The total energy state of each satellite $S_j \in \mathcal{S}$ evolves over time according to the background consumption, task execution energy, and solar charging. During the sunlight period of the orbit, a satellite can recharge its battery through solar power generation, whereas

during the eclipse period, it receives no solar power and must rely on the battery for background operation and data processing. The battery gets recharged during the sunlight period when the solar input exceeds energy consumption.

Task execution energy. When satellite S_j executes a fraction of job $\zeta_{i,m}$ in slot t_k , the corresponding task-execution energy is modeled as

$$E_{i,j,k} = P_j^{\text{cap}} T_{i,j}^{\text{cap}} + P_j^{\text{cmp}} T_i^{\text{cmp}} + P_j^{\text{dl}} T_{i,j}^{\text{dl}}, \quad (5)$$

where P_j^{cap} , P_j^{cmp} , and P_j^{dl} denote the average power of satellite S_j in capturing, computation, and downlink modes, respectively. Under the adopted slot-based abstraction, the total energy consumption of S_j in slot t_k is

$$E_{j,k}^{\text{slot}} = \sum_{i \in \mathcal{K}} \sum_{m \in \mathcal{M}_i} x_{i,j,k,m} E_{i,j,k}. \quad (6)$$

Solar charging. STK directly provides the solar illumination on each satellite as time-stamped intensity percentages. Let intensity $I_{j,k} \in [0, 100]$ denote the average solar intensity percentage received by satellite S_j during slot t_k . Let P_j^{max} be the rated solar panel power of S_j at 100% illumination, and let Δt be the slot length. The harvested energy in slot t_k is computed as

$$e_{j,k} = P_j^{\text{max}} \left(\frac{I_{j,k}}{100} \right) \Delta t. \quad (7)$$

The battery energy of satellite S_j then evolves according to

$$b_{j,k+1} = \min(B_j, b_{j,k} - E_{j,k}^{\text{slot}} + e_{j,k}), \quad (8)$$

where B_j is the battery capacity and $E_{j,k}^{\text{slot}}$ is the total energy consumption of S_j during slot t_k . The discretized energy values $e_{j,k}$ are produced during preprocessing and subsequently used to make scheduling decisions described in Section 3.

2.8 Optimization Problem

Due to limited visibility and energy availability, not all released jobs can be successfully executed. We define the ratio between the total completed workload and the total generated workload as the Coverage Ratio (CR). The optimization problem is formulated as

$$\max_x \frac{\sum_{i \in \mathcal{K}} \sum_{m \in \mathcal{M}_i} \sum_{j \in \mathcal{S}} \sum_{k \in \mathcal{T}} x_{i,j,k,m} C_{i,m}}{\sum_{i \in \mathcal{K}} \sum_{m \in \mathcal{M}_i} C_{i,m}} \quad (9a)$$

$$\text{s.t. } x_{i,j,k,m} \in [0, 1], \quad \forall i, j, m, k, \quad (9b)$$

$$x_{i,j,k,m} \leq a_{i,j,k}, \quad \forall i, j, m, k, \quad (9c)$$

$$x_{i,j,k,m} = 0, \quad \forall i, j, m, t_k \notin [r_{i,m}, r_{i,m} + D_{i,m}], \quad (9d)$$

$$\sum_{k \in \mathcal{T}} \sum_{j \in \mathcal{S}} x_{i,j,k,m} \leq 1, \quad \forall i, m, \quad (9e)$$

$$E_{j,k}^{\text{slot}} = e_b + \sum_{i \in \mathcal{K}} \sum_{m \in \mathcal{M}_i} x_{i,j,k,m} E_{i,j,k}, \quad \forall j, k, \quad (9f)$$

$$b_{j,k+1} = \min(B_j, b_{j,k} - E_{j,k}^{\text{slot}} + e_{j,k}), \quad \forall j, k. \quad (9g)$$

The formulation in (9) is interpreted as follows. The objective in (9a) maximizes the coverage ratio, i.e., the fraction of the total generated workload that can be completed over the planning horizon. Constraint (9b) defines the feasible range of the decision variable $x_{i,j,k,m}$, which represents the fraction of job $\zeta_{i,m}$ assigned to satellite S_j in slot t_k . Constraint (9c) enforces the geometric visibility requirement, allowing an assignment only if satellite S_j can observe task κ_i in slot t_k . Constraint (9d) imposes the release-deadline window of each job, so that no assignment is allowed outside $[r_{i,m}, r_{i,m} + D_{i,m}]$. Constraint (9e) ensures that each job is completed at most once by limiting the total assigned fraction to at most one. Constraint (9f) defines the total slot-level energy consumption of each satellite, including both the background energy e_b and the energy consumed by assigned jobs. Constraint (9g) models the battery dynamics across time slots, including energy consumption, harvested solar energy, and the battery-capacity limit.

3 Framework and Algorithm Design

We develop **Energy-Carryover Flow Scheduling (ECoFlow)**, a unified max-flow-based framework that maximizes the coverage ratio defined in Section 2.8 under jointly time-varying visibility and energy constraints. Unlike existing slot-wise methods that optimize only the current slot, ECoFlow reasons globally over the entire planning horizon and explicitly models inter-slot battery carryover as transferable flow in a time-expanded graph. This allows energy harvested in earlier slots to be preserved and used by later tasks, improving the utilization of limited onboard resources. The key idea is to convert time-varying orbital resource constraints into unified flow-conservation and capacity constraints, and then derive scheduling decisions through maximum-flow optimization.

Unifying workload in the time domain. To connect the scheduling objective with the flow-based formulation used in ECoFlow, we convert both task demand and satellite resource supply into a unified time metric. Specifically, for each job $\zeta_{i,m}$, we define its normalized workload time as

$$d_{i,m} = \frac{C_{i,m}}{f}, \quad (10)$$

where $C_{i,m}$ is the job workload measured in CPU cycles and f is the satellites' onboard CPU frequency.

Feasible assignment set. For each job $\zeta_{i,m}$, we define the set of feasible satellite-time pairs as

$$\Omega_{i,m} = \{(j, k) \mid a_{i,j,k} = 1, t_k \in [r_{i,m}, r_{i,m} + D_{i,m}]\}. \quad (11)$$

That is, a satellite-time pair (j, k) is feasible for job $\zeta_{i,m}$ if satellite S_j can observe the target of task κ_i in slot t_k and the slot lies within the release-deadline window of the job.

ECoFlow graph construction. Based on the normalized workload definition and the feasible assignment sets above, ECoFlow constructs a time-expanded graph, as illustrated in Figure 2. The graph contains six types of nodes arranged in six layers: (i) a source node src ; (ii) one task node for each task $\kappa_i \in K$; (iii) one job node for each released job $\zeta_{i,m} \in M_i$; (iv) one satellite-time node $u_{j,k}$ for each pair (S_j, t_k) ; (v) one energy node $q_{j,k}$ for each pair (S_j, t_k) ; and (vi) a sink node dst .

An edge from src to each task node κ_i is assigned capacity $\sum_{m \in M_i} d_{i,m}$, representing the total normalized workload released

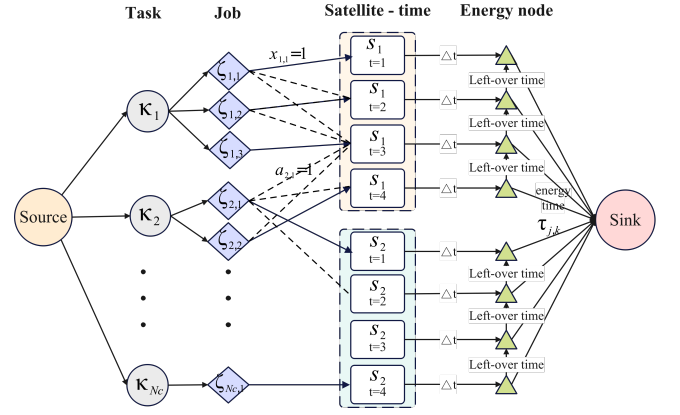


Figure 2: ECoFlow graph construction with inter-slot energy carry-over.

by task κ_i over the planning horizon. An edge from each task node κ_i to its job node $\zeta_{i,m}$ is assigned capacity $d_{i,m}$, representing the normalized workload demand of that job.

For each feasible pair $(j, k) \in \Omega_{i,m}$, ECoFlow adds an edge from job node $\zeta_{i,m}$ to satellite-time node $u_{j,k}$ with capacity Δt . This means that job $\zeta_{i,m}$ can receive at most one slot of service from satellite S_j in slot t_k , while a job may accumulate service from multiple feasible satellites and slots before its deadline. In this way, the graph naturally supports fractional workload assignment across satellites and time slots.

Each satellite-time node $u_{j,k}$ further connects to its corresponding energy node $q_{j,k}$ with capacity Δt , which captures the per-slot execution-time limit of the satellite.

Inter-slot energy carryover in ECoFlow. The key novelty of ECoFlow is that inter-slot battery evolution is embedded directly into the graph as transferable flow, rather than handled by a myopic slot-by-slot rule. Before graph construction, we reserve the mandatory background consumption e_b according to the energy dynamics in Section 2.7, and denote by $\bar{e}_{j,k}$ the residual energy in slot t_k that can be used for task execution. Let ψ denote the energy cost of one CPU cycle. Since the CPU frequency is f , one second of computation consumes ψf joules. Therefore, the residual energy can be converted into time-equivalent service capacity as

$$\tau_{j,1}^{in} = \frac{b_{j,1} + \bar{e}_{j,1}}{\psi f}; \quad \tau_{j,k}^{in} = \frac{\bar{e}_{j,k}}{\psi f}, \quad \text{for } k \geq 2, \quad (12)$$

where $b_{j,1}$ is the initial battery state of satellite S_j . Similarly, the battery capacity is converted into time units as

$$\tau_j^B = \frac{B_j}{\psi f}. \quad (13)$$

For each energy node $q_{j,k}$, ECoFlow adds an edge $(q_{j,k}, dst)$ with capacity $\tau_{j,k}^{in}$ to represent the new energy injected into the graph by solar harvesting and the initial battery state. Moreover, for every $k \geq 2$, ECoFlow adds a carryover edge $(q_{j,k}, q_{j,k-1})$ with capacity τ_j^B . The capacity τ_j^B ensures that no more than the battery capacity can be transferred across any slot boundary. Thus, cross-slot energy evolution becomes a flow-constrained resource in the graph, enabling ECoFlow to optimize over the full planning horizon rather than make locally optimal slot-wise decisions.

Coverage-maximizing formulation and the ECoFlow algorithm. Let $f(\zeta_{i,m}, u_{j,k})$ denote the flow on the edge from job node $\zeta_{i,m}$ to satellite-time node $u_{j,k}$. Since each unit of flow represents one unit of normalized workload time, the value of an *src-dst* flow is exactly the total covered normalized workload time. Accordingly, the scheduling variable in Section 2.6 can be recovered from the flow as

$$x_{i,j,k,m} = \frac{f(\zeta_{i,m}, u_{j,k})}{d_{i,m}}. \quad (14)$$

Let F^* denote the maximum flow value in the constructed graph. Then, the coverage ratio can be equivalently written as

$$\text{CR} = \frac{\sum_{i \in K} \sum_{m \in M_i} \sum_{j \in S} \sum_{k \in T} x_{i,j,k,m} C_{i,m}}{\sum_{i \in K} \sum_{m \in M_i} C_{i,m}} = \frac{F^*}{\sum_{i \in K} \sum_{m \in M_i} d_{i,m}}. \quad (15)$$

Hence, maximizing the flow value in the constructed graph is equivalent to maximizing the original coverage ratio.

Based on this equivalence, ECoFlow first preprocesses the feasible assignment sets $\Omega_{i,m}$ and the residual task-execution energy profile $\bar{e}_{j,k}$, and then constructs the time-expanded flow graph described above. It then applies the Push-Relabel maximum-flow algorithm [11], which is known to scale better than classical augmenting-path methods on such large max-flow instances, to compute the maximum executable normalized workload time F^* . Finally, the scheduling decision is recovered from the flow on the job-to-satellite-time edges according to eq. (14), and the resulting coverage ratio is computed by eq. (15). In this way, ECoFlow directly returns both the maximum covered workload and the corresponding scheduling decision under jointly time-varying visibility and energy constraints. Algorithm 1 summarizes the detailed procedure of ECoFlow.

Complexity Analysis. Let $N_J = \sum_{\kappa_i \in K} |M_i|$ denote the total number of jobs released over the planning horizon, and let $\mathcal{G} = (\mathcal{N}, \mathcal{L})$ denote the graph constructed by ECoFlow. The graph contains task nodes, job nodes, satellite-time nodes, energy nodes, together with the source and sink, so the total number of nodes is

$$|\mathcal{N}| = 2 + N_c + N_J + 2N_s N_t. \quad (16)$$

Let $\Omega = \bigcup_{i,m} \Omega_{i,m}$ be the set of all feasible job-to-satellite-time pairs. The edge set includes source-to-task, task-to-job, job-to-satellite-time, satellite-time-to-energy, energy-to-sink, and inter-slot carryover edges, yielding

$$|\mathcal{L}| = N_c + N_J + |\Omega| + 2N_s N_t + N_s (N_t - 1). \quad (17)$$

In the worst case, $|\Omega| = O(N_J N_s N_t)$, so constructing the ECoFlow graph requires $O(|\mathcal{N}| + |\mathcal{L}|)$ time and space. The maximum-flow computation is then performed using the Push-Relabel algorithm, whose standard worst-case running time determines the overall complexity

$$O(|\mathcal{N}|^2 |\mathcal{L}|) = O((N_c + N_J + N_s N_t)^2 \cdot N_J N_s N_t). \quad (18)$$

The recovery of $x_{i,j,k,m}$ from the final flow is linear in the number of job-to-satellite-time edges and therefore does not change the overall complexity.

Algorithm 1: ECoFlow Scheduling Algorithm

Input: Task set K , satellite set S , time slots T , visibility tensor A , initial battery states $b_{j,1}$, harvested energy profile $e_{j,k}$
Output: Scheduling decision $x_{i,j,k,m}$, maximum flow value F^* , and coverage ratio CR

- 1 Compute the residual task-execution energy $\bar{e}_{j,k}$ by reserving the background energy consumption e_b ;
- 2 Compute the normalized workload time $d_{i,m} = C_{i,m}/f$ for each released job $\zeta_{i,m}$;
- 3 Construct the flow graph with source *src*, sink *dst*, task nodes, job nodes, satellite-time nodes $u_{j,k}$, and energy nodes $q_{j,k}$;
- 4 **foreach** task $\kappa_i \in K$ **do**
- 5 Add edge (*src*, κ_i) with capacity $\sum_{m \in M_i} d_{i,m}$;
- 6 **foreach** job $\zeta_{i,m} \in M_i$ **do**
- 7 Add edge (κ_i , $\zeta_{i,m}$) with capacity $d_{i,m}$;
- 8 Compute the feasible set $\Omega_{i,m}$ according to A , $r_{i,m}$, and $D_{i,m}$;
- 9 **foreach** $(j, k) \in \Omega_{i,m}$ **do**
- 10 Add edge ($\zeta_{i,m}$, $u_{j,k}$) with capacity Δt ;
- 11 **end**
- 12 **end**
- 13 **end**
- 14 **foreach** satellite S_j and time slot t_k **do**
- 15 Add edge ($u_{j,k}$, $q_{j,k}$) with capacity Δt ;
- 16 **if** $k = 1$ **then**
- 17 Add edge ($q_{j,1}$, *dst*) with capacity $(b_{j,1} + \bar{e}_{j,1})/(\psi f)$;
- 18 **else**
- 19 Add edge ($q_{j,k}$, *dst*) with capacity $\bar{e}_{j,k}/(\psi f)$;
- 20 Add carryover edge ($q_{j,k}$, $q_{j,k-1}$) with capacity $B_j/(\psi f)$;
- 21 **end**
- 22 **end**
- 23 Run Push-Relabel on the constructed graph to obtain F^* ;
- 24 Recover $x_{i,j,k,m} = f(\zeta_{i,m}, u_{j,k})/d_{i,m}$ and compute $\text{CR} = F^* / \sum_{i \in K} \sum_{m \in M_i} d_{i,m}$;
- 25 **return** $x_{i,j,k,m}$, F^* , CR;

4 Experimental Evaluation

4.1 Experiment Setup

Evaluation parameters. Our experimental settings follow typical orbital edge computing configurations from prior work, particularly the parameter choices in [12]. We consider an LEO Earth-observation constellation and generate real satellite visibility traces over a 24-hour planning horizon for all scheduling evaluations. The monitoring workload consists of multiple geographically distributed ground targets and reflects time-sensitive observation demands as discussed in the introduction. Each task periodically releases jobs. Detailed experimental parameters are summarized in Table 1.

Compared algorithms. We compare ECoFlow with the following baselines:

- **Earliest Deadline First (EDF):** A deadline-driven baseline motivated by prior EDF-based scheduling studies for energy-harvesting real-time systems [13]. Jobs are prioritized by earliest deadlines, and feasible satellite-time opportunities are allocated without explicitly considering residual energy.

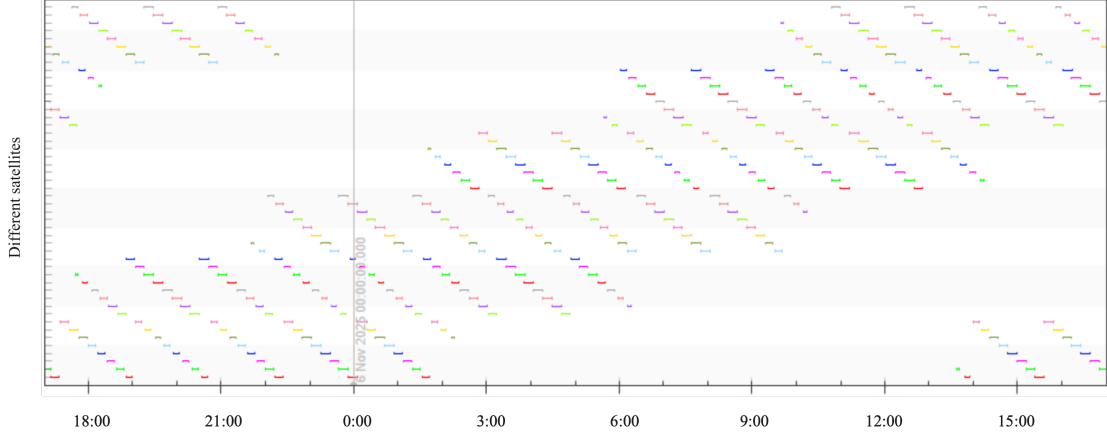


Figure 3: STK-based visibility analysis showing satellite-to-task access intervals over time.

Table 1: Experimental parameters.

Parameter	Value
Number of satellites	136
Number of ground monitoring locations	6
Visibility trace duration	24 h
Energy per CPU cycle	1×10^{-8} J
CPU cycles per second	1×10^8 cycles/s
Battery capacity	600 s
Solar panel power	12 W
Time-slot length	60 s
Average task period	200 s
Average task deadline	70 s
Average job execution time (reference)	50 s

row corresponds to one satellite, and all access intervals of that satellite are drawn using the same color. Different satellites are assigned different colors. Within each row, the short colored bars represent the time intervals during which that satellite has line-of-sight visibility to the target area. In our evaluation, these access intervals are exported from STK and discretized into the per-slot sensing and downlink availability used by the scheduler.

Figure 4 shows the solar illumination profile of Satellite 1 on Orbit 1 over a 6-hour window. The satellite exhibits a highly periodic sunlight-eclipse pattern, with a cycle length of approximately 1.5 hours, which is consistent with the orbital period of a typical LEO satellite. Within each cycle, the satellite receives sunlight for roughly 1 hour, followed by a ~ 0.5 -hour eclipse interval in which the illumination drops to zero. This periodic light-dark transition directly determines the available solar charging opportunities, as the satellite can recharge only during the illuminated segment of each orbit. Consequently, the battery must sustain all energy consumption during the eclipse phase. In our evaluation, the illumination trace is converted into per-slot harvested-energy inputs for the energy model.

4.2 Algorithm Evaluation

Figure 5 reports the coverage ratio under varying constellation and task parameters. Across almost the entire range, ECoFlow consistently performs best, followed by Energy-first EDF, EDF, and Random. This ordering indicates that jointly accounting for deadline feasibility, visibility windows, and inter-slot energy carryover is more effective than local greedy or random assignment.

Figure 5(a) shows the coverage ratio versus average deadline. Smaller deadlines imply tighter feasible windows and thus lower coverage. At 20 s, all four methods perform similarly, with coverage around 7%, because many jobs expire before finding feasible satellite-time assignments. As the average deadline increases from 20 s to 40 s, the coverage ratio rises for all methods, with ECoFlow remaining the best. Its advantage is most pronounced in the medium-deadline range: at 30 s, ECoFlow reaches about 0.57, compared with 0.51 for Energy-first EDF, 0.47 for EDF, and

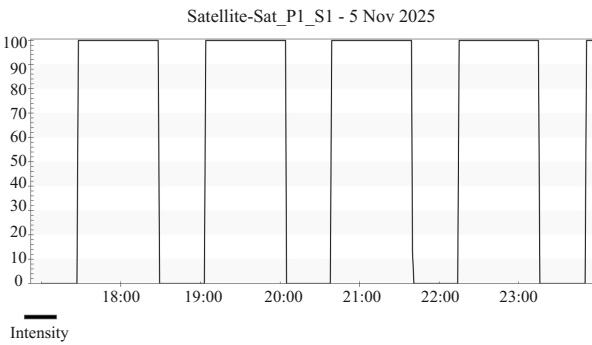


Figure 4: Solar power profile of a satellite generated by STK.

- **Energy-first EDF:** An energy-aware extension of EDF used as a heuristic baseline in our setting. It first prioritizes jobs by earliest deadlines and then prefers feasible satellites with higher residual energy.
- **Random:** An uninformed baseline that assigns jobs to randomly selected feasible satellite-time opportunities until the job is completed or no feasible opportunity remains.

STK-Based Visibility and Power Analysis. Figure 3 illustrates an example of the STK-based visibility analysis. Each horizontal

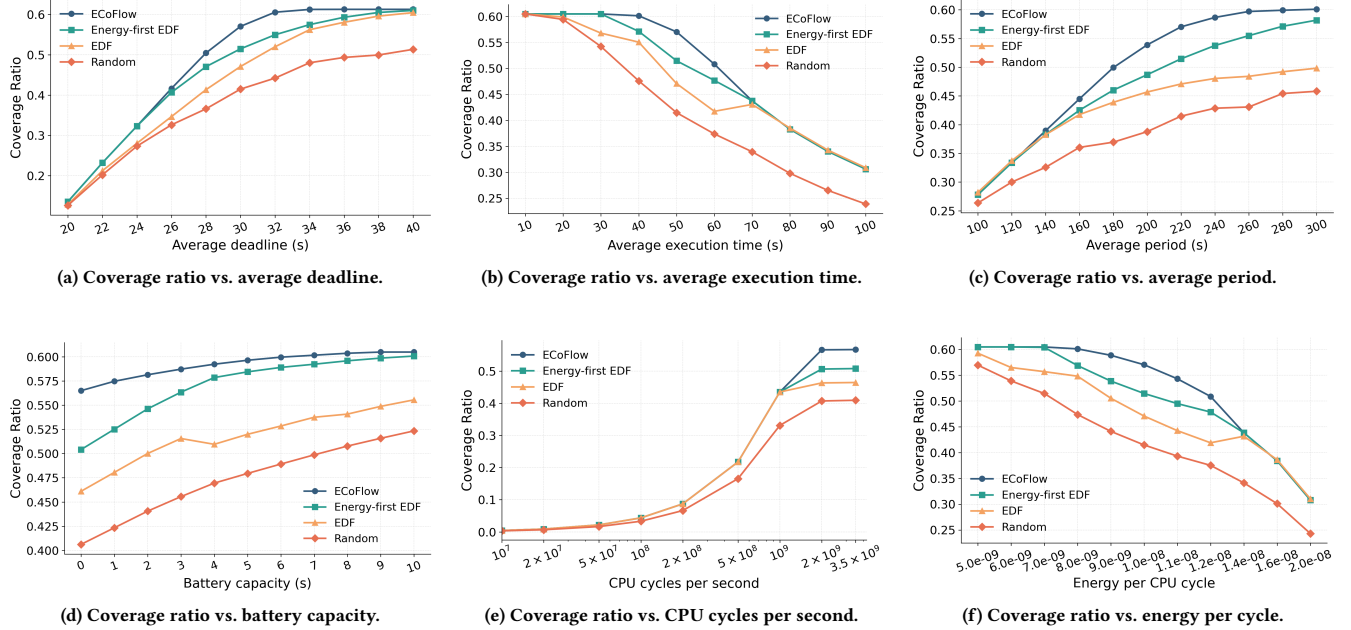


Figure 5: Coverage ratio under six representative timing, computation, and energy-related parameters.

0.41 for Random, corresponding to improvements of about 11.8%, 21.3%, and 39.0%, respectively. This reflects the benefit of globally coordinating visibility, energy, and timing constraints. Energy-first EDF also consistently outperforms EDF, confirming the importance of energy awareness. When the average deadline exceeds about 34–40 s, the curves flatten, suggesting that deadline is no longer the dominant bottleneck.

In Figure 5(b), the coverage ratio of all four methods decreases as the average execution time increases. When execution time is 10–20 s, the four curves nearly overlap, indicating that tasks are light and the system is not yet strongly constrained by computation or energy. As execution time increases to 20–40 s, the curves begin to separate: EDF and Random decline earlier, while Energy-first EDF remains closer to ECoFlow, showing that energy-aware scheduling becomes beneficial once the workload is no longer trivially schedulable. The gap is most pronounced in the 40–70 s range, where ECoFlow consistently performs best, followed by Energy-first EDF, EDF, and Random, due to its global coordination over the planning horizon. When execution time further increases to 80–100 s, ECoFlow, Energy-first EDF, and EDF gradually converge, while Random remains clearly worse, indicating that execution time itself becomes the dominant bottleneck in this heavily resource-constrained regime.

Figure 5(c) shows that all four curves increase as the average period grows from 100 s to 300 s. Smaller periods correspond to denser task arrivals and thus lower coverage, since more jobs cannot be completed in time. As the period increases, system load decreases, and scheduling becomes easier. In the 100–140 s range, all methods remain strongly constrained by frequent arrivals. In the 160–240 s range, the curves separate clearly, with ECoFlow improving the fastest and remaining the best, followed by Energy-first EDF, EDF,

and Random. This suggests that as the system gains more flexibility, the ability to exploit available resources becomes increasingly important. When the period reaches 260–300 s, the curves begin to flatten, indicating diminishing returns and suggesting that the average period is no longer the sole dominant bottleneck.

Figure 5(d) shows the coverage ratio versus battery capacity. All four methods improve as battery capacity increases, since satellites can preserve more harvested solar energy and sustain task execution during eclipse periods. ECoFlow consistently performs best, with its advantage being most evident in the low-capacity regime. Energy-first EDF also clearly outperforms EDF, indicating that energy awareness is particularly important when battery capacity is limited. In the high-capacity range, the curves gradually flatten, suggesting diminishing returns from further battery expansion.

Figure 5(e) shows the coverage ratio versus CPU cycles per second. All four methods benefit from higher CPU speed. When CPU speed is below 5×10^7 cycles/s, the coverage ratio remains close to zero because computation is insufficient to complete most jobs before their deadlines. As CPU speed increases from 10^8 to 10^9 cycles/s, the coverage ratio rises rapidly for all methods, indicating that computation becomes less restrictive. ECoFlow consistently achieves the highest coverage ratio, while Energy-first EDF remains above EDF once CPU speed reaches 10^8 cycles/s, showing that energy-aware scheduling still matters after computational capacity improves. Random also improves with CPU speed, but remains the worst. When CPU speed exceeds 2×10^9 cycles/s, the curves flatten, indicating that computation is no longer the main bottleneck and that coverage is instead limited by energy and deadline constraints.

Figure 5(f) shows the coverage ratio versus energy per CPU cycle, which reflects the computation-energy efficiency of the onboard platform. Overall, the coverage ratio of all methods decreases as

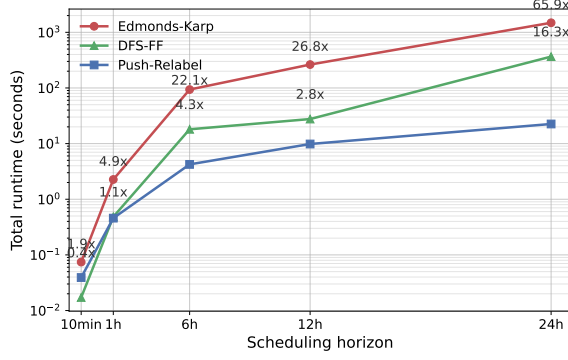


Figure 6: Runtime comparison of three exact max-flow solvers under different scheduling horizons.

computation becomes more energy-intensive. When the energy per CPU cycle is between 5×10^{-9} and 7×10^{-9} J, all methods maintain relatively high coverage, with ECoFlow and Energy-first EDF both around 0.60. As it increases from 8×10^{-9} to 1.2×10^{-8} J, the coverage ratio declines steadily for all methods, and the advantage of ECoFlow becomes more pronounced. Energy-first EDF also consistently outperforms EDF in this range, indicating that energy awareness remains beneficial as computation cost rises, while Random stays below the other three methods. When the energy per CPU cycle exceeds 1.4×10^{-8} J, the coverage ratio drops further for all methods. By 2×10^{-8} J, ECoFlow, Energy-first EDF, and EDF converge to nearly the same level, whereas Random remains clearly worse, suggesting that energy availability becomes the dominant bottleneck under very high computation energy cost.

Figure 6 compares the runtime of three exact max-flow solvers: Edmonds-Karp [14], DFS-based Ford-Fulkerson (DFS-FF) [15], and Push-Relabel, under different scheduling horizons. For the smallest instance (10 min), DFS-FF is the fastest, while Push-Relabel and Edmonds-Karp incur slightly higher overhead. As the horizon increases, however, Push-Relabel quickly becomes the most efficient and scalable method. At 1 h, 6 h, 12 h, and 24 h, DFS-FF requires about 1.1x, 4.3x, 2.8x, and 16.3x the runtime of Push-Relabel, while Edmonds-Karp requires about 4.9x, 22.1x, 26.8x, and 65.9x. This trend is explained by the structure of the scheduling graph: although it has only a few layers, its width grows rapidly with the horizon due to the increasing numbers of jobs, satellite-time nodes, energy nodes, and feasible job-to-slot edges. In this setting, the breadth-first search in Edmonds-Karp becomes increasingly costly, whereas DFS-FF is lighter-weight in our implementation. Push-Relabel, by contrast, handles large residual networks more efficiently and thus scales much better with problem size. Even for a 24-hour horizon, its runtime remains only on the order of tens of seconds, which is still practical for offline feasibility checking. These results support the practical applicability of ECoFlow and justify our use of Push-Relabel as the max-flow solver.

5 Related Work

Existing studies have investigated satellite scheduling, task allocation, and real-time resource coordination from several complementary perspectives. In satellite systems, prior research has explored the joint optimization of service deployment and task scheduling for improving long-term system performance [16], has developed

two-stage approaches for task decision and allocation in large-scale satellite networks [17], and has adopted graph-based formulations to characterize task dependencies and conflicts across multiple satellites [18]. Related work has further examined the scheduling of computing and communication resources in dynamic satellite and edge environments, with the goals of reducing execution time, communication latency, and energy consumption [19–22]. From a networking perspective, Li et al. have proposed TinyLEO to address demand-aware resource matching and topology management in highly dynamic LEO networks [23]. Chou et al. have leveraged collaborative commercial ground stations and 5G infrastructure to reduce energy consumption and latency while extending satellite lifetime [24].

More closely related to our problem, the authors in [25] have studied task allocation for real-time remote-sensing pipelines on LEO satellite constellations and have proposed an efficient heuristic with provable performance bounds. In a broader real-time energy-aware setting, the authors in [26] have shown that schedulability analysis should explicitly account for time-varying energy harvesting and fluctuating computational capacity. These studies have highlighted the importance of incorporating dynamic resource availability into scheduling decisions.

While these assumptions have helped simplify scheduling design and have enabled important progress in satellite and real-time resource coordination, they are less suited to sustainable in-orbit analytics for Earth observation, where target visibility, task timing, onboard computation, and energy availability evolve jointly over time. Motivated by this setting, our work develops a unified model and an energy-aware flow-based scheduler that explicitly captures orbital visibility and battery carryover across time slots.

6 Conclusion

This paper investigated sustainable in-orbit analytics in LEO Earth-observation constellations under continuously changing visibility and energy conditions. A central observation is that, in such systems, scheduling quality depends not only on whether a task is currently visible or urgent, but also on how current energy usage affects future feasibility over the planning horizon. To capture this inter-temporal dependency, we proposed *Energy-Carryover Flow Scheduling (ECoFlow)*, a unified discrete-time scheduling framework that jointly models periodic monitoring jobs, release-deadline constraints, satellite-target visibility, onboard computation, and battery evolution, while explicitly representing harvested energy and battery carryover in a time-expanded graph. This formulation enables ECoFlow to reason globally over time rather than making myopic slot-wise decisions. Experimental results based on TLE-propagated orbital traces and representative monitoring workloads show that ECoFlow consistently achieves a higher coverage ratio than the baseline methods, with improvements of up to 12% under diverse timing, computation, and energy settings. Overall, the results demonstrate that explicitly incorporating energy evolution into the scheduling formulation is an effective way to support sustained in-orbit analytics under realistic orbital dynamics. An important direction for future work is to extend ECoFlow to heterogeneous satellite platforms with different computing capabilities, battery capacities, and energy-harvesting characteristics.

References

- [1] X. Liu, N. Iftikhar, and X. Xie, "Survey of real-time processing systems for big data," in *Proceedings of the 18th International Database Engineering & Applications Symposium*, ser. IDEAS '14, 2014, pp. 356–361.
- [2] T. Ahmad, A. Morel, N. Cheng, K. Palaniappan, P. Calyam, K. Sun, and J. Pan, "Future uav/drone systems for intelligent active surveillance and monitoring," *ACM Computing Surveys*, vol. 58, no. 2, pp. 1–37, 2025.
- [3] G. Peng, S. Wang, T. Huang, F. Li, K. Zhao, Y. Huang, and Z. Xiong, "Fastts: Enabling fault-tolerant and time-sensitive scheduling in space-terrestrial integrated networks," *IEEE Journal on Selected Areas in Communications*, vol. 42, no. 12, pp. 3551–3565, 2024.
- [4] P. Zhang, Q. Qin, S. Zhang, X. Zhao, X. Yan, W. Wang, and H. Zhang, "Near real-time remote sensing based on satellite internet: Architectures, key techniques, and experimental progress," *Aerospace*, vol. 11, pp. 167–185, 2024.
- [5] D. Zhou, M. Sheng, C. Bao, Y. Wang, J. Li, and Z. Han, "Mission-driven resource scheduling in satellite-terrestrial networks: From perspective of collaboration and reconfiguration," *IEEE Transactions on Communications*, vol. 73, no. 8, pp. 6705–6719, 2025.
- [6] O. Kodheli, E. Lagunas, N. Maturo, S. K. Sharma, B. Shankar, J. F. M. Montoya, J. C. M. Duncan, D. Spano, S. Chatzinotas, S. Kisseleff, J. Querol, L. Lei, T. X. Vu, and G. Goussetis, "Satellite communications in the new space era: A survey and future challenges," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 1, pp. 70–109, 2021.
- [7] M. Centenaro, C. E. Costa, F. Granelli, C. Sacchi, and L. Vangelista, "A survey on technologies, standards and open challenges in satellite iot," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1693–1720, 2021.
- [8] Y. Huang, X. Jiang, S. Chen, F. Yang, and J. Yang, "Pheromone incentivized intelligent multipath traffic scheduling approach for leo satellite networks," *IEEE Transactions on Wireless Communications*, vol. 21, no. 8, pp. 5889–5902, 2022.
- [9] Analytical Graphics, Inc., "Systems Tool Kit (STK)," <https://www.agi.com/products/stk>, 2024, accessed: 2025-11-18.
- [10] J. Zhou, J. Liang, L. Zhao, S. Wan, H. Cai, and F. Xiao, "Latency-energy efficient task offloading in the satellite network-assisted edge computing via deep reinforcement learning," *IEEE Transactions on Mobile Computing*, vol. 24, no. 4, pp. 2644–2659, 2025.
- [11] A. V. Goldberg and R. E. Tarjan, "A new approach to the maximum-flow problem," *Journal of the ACM*, vol. 35, no. 4, pp. 921–940, 1988.
- [12] Y. Chen, Q. Zhang, R. Xing, Y. Li, X. Ma, Y. Zhang, A. Zhou, and S. Wang, "Slice: Energy-efficient satellite-ground co-inference via layer-wise scheduling optimization," *IEEE Transactions on Services Computing*, vol. 18, no. 4, pp. 2388–2402, 2025.
- [13] M. Chetto and A. Queudet, "A note on edf scheduling for real-time energy harvesting systems," *IEEE Transactions on Computers*, vol. 63, no. 4, pp. 1037–1040, 2014.
- [14] J. Edmonds and R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems," *Journal of the ACM*, vol. 19, no. 2, pp. 248–264, 1972.
- [15] L. R. Ford and D. R. Fulkerson, "Maximal flow through a network," *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956.
- [16] Q. Tang, R. Xie, Z. Fang, T. Huang, T. Chen, R. Zhang, and F. R. Yu, "Joint service deployment and task scheduling for satellite edge computing: A two-timescale hierarchical approach," *IEEE Journal on Selected Areas in Communications*, vol. 42, no. 5, pp. 1063–1079, 2024.
- [17] G. Li, X. Li, J. Li, J. Chen, and X. Shen, "Ptmb: An online satellite task scheduling framework based on pre-trained markov decision process for multi-task scenario," *Knowledge-Based Systems*, vol. 284, pp. 1–13, 2024.
- [18] P. Li, J. Li, H. Li, S. Zhang, and G. Yang, "Graph based task scheduling algorithm for earth observation satellites," in *2018 IEEE Global Communications Conference*, 2018, pp. 1–7.
- [19] Q. Jiang, L. Zheng, Y. Zhou, H. Liu, Q. Kong, Y. Zhang, and B. Chen, "Efficient on-orbit remote sensing imagery processing via satellite edge computing resource scheduling optimization," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 63, pp. 1–19, 2025.
- [20] D. Li, S. Wu, Y. Wang, W. Wu, and Q. Zhang, "Intelligent task scheduling in hybrid geo-leo satellite-assisted marine iot network," *IEEE Internet of Things Journal*, vol. 12, no. 7, pp. 8353–8367, 2025.
- [21] G. Wang, F. Yang, J. Song, and Z. Han, "Resource allocation and load balancing for beam hopping scheduling in satellite-terrestrial communications: A cooperative satellite approach," *IEEE Transactions on Wireless Communications*, vol. 24, no. 2, pp. 1339–1354, 2025.
- [22] Z. Lin, Z. Ni, L. Kuang, C. Jiang, and Z. Huang, "Satellite-terrestrial coordinated multi-satellite beam hopping scheduling based on multi-agent deep reinforcement learning," *IEEE Transactions on Wireless Communications*, vol. 23, no. 8, pp. 10 091–10 103, 2024.
- [23] Y. Li, Y. Chen, J. Yang, J. Zhang, B. Sun, L. Liu, H. Li, J. Wu, Z. Lai, Q. Wu, and J. Liu, "Small-scale leo satellite networking for global-scale demands," in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 917–931.
- [24] Y. Chou, C. Long, H. Wang, H. Fang, H. Zhao, M. Zhang, X. Fan, and J. Liu, "Commercial dishes can be my ladder: Sustainable and collaborative data offloading in leo satellite networks," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025, pp. 1–10.
- [25] M. Lv, X. Peng, W. Xie, and N. Guan, "Task allocation for real-time earth observation service with leo satellites," in *Proceedings of the IEEE Real-Time Systems Symposium*, 2022, pp. 185–197.
- [26] Z. Ma, Z. Yan, H. Wang, and N. Guan, "Energy-feasible real-time scheduling on satellites with orbit-dependent energy harvesting," in *37th Euromicro Conference on Real-Time Systems*, ser. Leibniz International Proceedings in Informatics, vol. 304, 2025, pp. 4:1–4:22.